



University of Frontier Technology, Bangladesh
Department of Internet of Things and Robotics Engineering

Course Code	CC 484
Course Title	Cloud Computing Sessional

Lab Mid Project Report

Dockerized To-Do Application

Submitted To

Farzana Akter

Assistant Professor

Department of Internet of Things and Robotics Engineering

University of Frontier Technology, Bangladesh

Submitted By

Sl	Class Roll	Name	Session
01	2201022	Mst. Esat Jahan Akhi	2022-2023
02	2201021	Most. Sukria Akter	2022-2023

Date:6 June, 2026

Docker Commands Used

1. Build Docker Image

```
docker build -t todo-app .
```

Purpose: Used to create a Docker image from the application source code and Dockerfile.

2. Run Docker Container

```
docker run -d -p 5000:5000 --name todo_container todo-app
```

Purpose: Used to create and run the Docker container.

3. Verify Running Container

```
docker ps
```

Purpose: Used to verify that the container is running successfully.

4. View Container Logs

```
docker logs todo_container
```

Purpose: Used to monitor application activity and logs inside the container.

5. Stop Docker Container

```
docker stop todo_container
```

Purpose: Used to stop the running Docker container safely.

Figure 1: Docker Image Build

```
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$ docker build -t todo-app .
[+] Building 2.6s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile               0.1s
=> == transferring dockerfile: 201B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.1s
=> [internal] load .dockerignore                                  0.1s
=> == transferring context: 2B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e915 0.2s
=> == resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e915 0.2s
=> [internal] load build context                                  0.3s
=> == transferring context: 141.95kB                              0.3s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                            0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY . .                                           0.0s
=> exporting to image                                              0.5s
=> == exporting layers                                             0.0s
=> == exporting manifest sha256:59ab370b2ed1f88c047ccdf2373e3a310732ddcb 0.0s
=> == exporting config sha256:a64bef4ac95f7a5a27ababb6b750b02c15619652dd 0.0s
=> == exporting attestation manifest sha256:5ea5fd60e9d9e31e416f490f754a 0.1s
=> == exporting manifest list sha256:c9862f1093f561be5179a974afb076981b5 0.1s
=> == naming to docker.io/library/todo-app:latest                 0.0s
=> == unpacking to docker.io/library/todo-app:latest               0.1s
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$
```

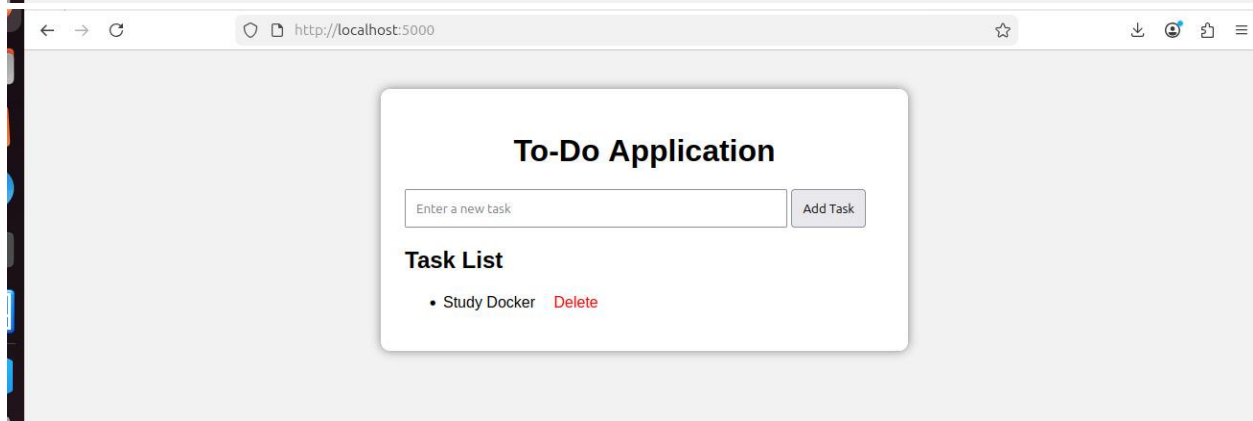
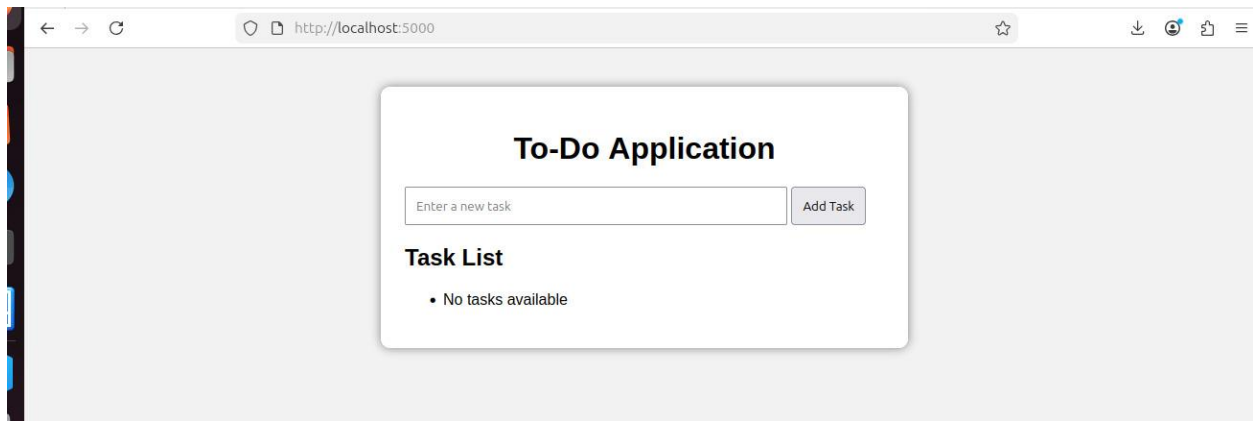
Figure 2: Running Docker Container

```
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$ docker run -d -p 5000:5000 --name todo_container todo-app
bf78454794f0f196bb3592014002c952daa01b21adc16774d7ebc31d26a53133
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
bf78454794f0   todo-app  "python app.py"         16 seconds ago Up 15 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  todo_
container
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$
```

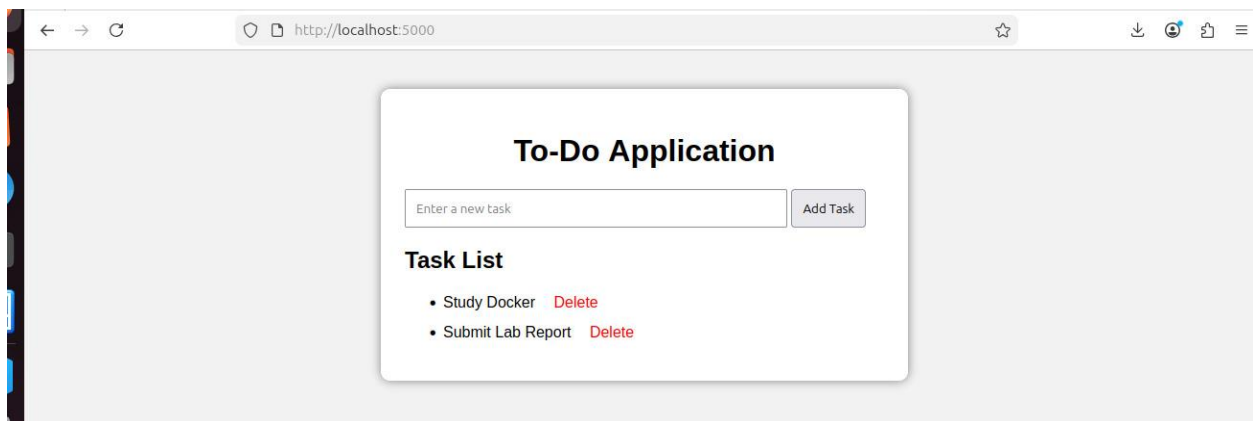
Figure 3: To-Do Application Output

<http://localhost:5000>

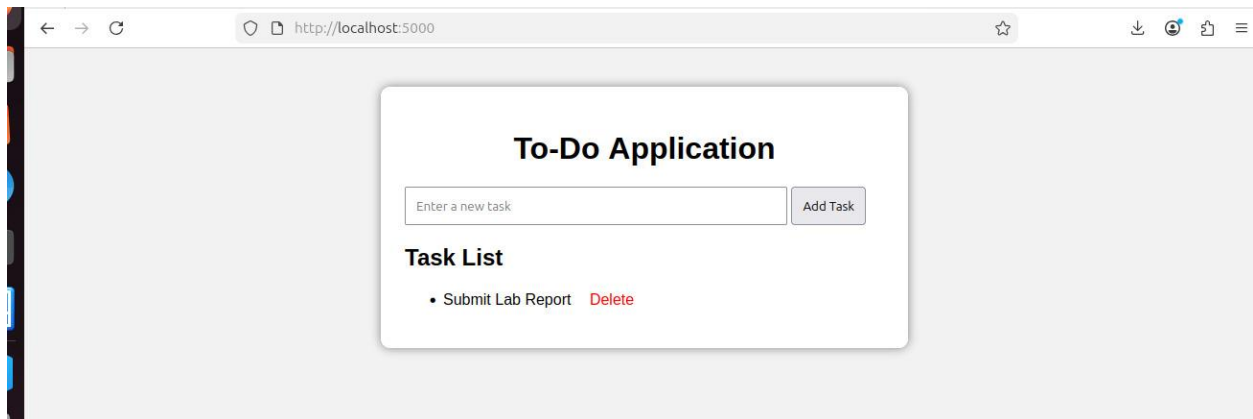
View task:



Adding task:



Deleting task:



Running Container:

Docker logs:

```
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$ docker logs todo_container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [09/Jun/2026 14:53:56] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [09/Jun/2026 14:53:56] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [09/Jun/2026 14:55:55] "POST /add HTTP/1.1" 302 -
172.17.0.1 - - [09/Jun/2026 14:55:55] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [09/Jun/2026 14:57:00] "POST /add HTTP/1.1" 302 -
172.17.0.1 - - [09/Jun/2026 14:57:00] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [09/Jun/2026 14:57:46] "GET /delete/0 HTTP/1.1" 302 -
172.17.0.1 - - [09/Jun/2026 14:57:46] "GET / HTTP/1.1" 200 -
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$
```

Docker ps:

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [09/Jun/2026 14:53:56] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [09/Jun/2026 14:53:56] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [09/Jun/2026 14:55:55] "POST /add HTTP/1.1" 302 -
172.17.0.1 - - [09/Jun/2026 14:55:55] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [09/Jun/2026 14:57:00] "POST /add HTTP/1.1" 302 -
172.17.0.1 - - [09/Jun/2026 14:57:00] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [09/Jun/2026 14:57:46] "GET /delete/0 HTTP/1.1" 302 -
172.17.0.1 - - [09/Jun/2026 14:57:46] "GET / HTTP/1.1" 200 -
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
bf78454794f0   todo-app   "python app.py"         11 minutes ago Up 11 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  todo_
container
(venv) esat-akhi@esat-akhi-VirtualBox:~/todo-docker-lab$
```

Question answer solution:

What is the difference between Docker Image and Docker Container?

Answer:

A Docker Image is a read-only template that contains the application code, dependencies, libraries, and configuration required to run an application. A Docker Container is a running instance of a Docker Image. An image acts as a blueprint, while a container is the actual execution environment created from that image.

Feature	Docker Image	Docker Container
Analogy	The Blueprint (Class)	The Building (Object/Instance)
Nature	A logical entity; a read-only template.	A real-world, runnable entity; a process.
Mutability	Immutable. Cannot be changed once built.	Mutable (temporarily). It has a read-write layer for runtime changes.
Resources	Does not consume CPU/RAM when stored (only disk space).	Consumes CPU and RAM only when running.
Creation	Created by building a Dockerfile.	Created by running an image using the docker run command.
Sharing	Can be pushed to and pulled from Docker Hub.	Cannot be shared directly. You must "commit" a container into a new image to share its state.
Layers	Made up of multiple read-only layers.	Made up of image layers plus one top read-write layer.
Connectivity	Cannot connect to or enter an image.	You can attach shell sessions (docker exec) to a running container.
Existence	Can exist independent of containers.	Cannot exist without a base image.

Question 2

What advantages does Docker provide for deploying the To-Do application?

Answer:

Docker provides several advantages for deployment:

1. It ensures consistency across different environments.
2. It simplifies deployment and dependency management.
3. It improves portability because the application can run on any system where Docker is installed.

Question 3

If another computer wants to run your application, what files are required and why?

Answer:

Another computer requires the project source files, including `app.py`, `Dockerfile`, `requirements.txt`, and `templates/index.html`.

- `app.py` contains the application logic.
- `Dockerfile` contains instructions for building the Docker image.
- `requirements.txt` contains the required Python dependencies.
- `templates/index.html` contains the user interface.

These files are necessary because Docker uses them to build the image and run the application correctly on another system.